

Credit Risk & Lending Policy Analysis

Foundation & Architecture Specification

Scoped v1 — hardened per council review. Data-analytics portfolio project targeting junior credit / risk / BI analyst roles.

Team	Two contributors (macOS + Windows)
Timeline	~2–3 weeks part-time (build ~1 wk, harden + BI + memo ~1–2 wk)
Dataset	Lending Club accepted loans (Kaggle mirror), 36-month term, mature vintages only
Core stack	Python 3.11 · DuckDB · scikit-learn · Streamlit · Power BI
Primary deliverables	PD model + calibration · Expected-Loss engine · Lending Policy Simulator · 2-page Power BI · 1-page memo
Status	Design locked; foundation fixes incorporated (see §2)

This document specifies *what is built, on what it runs, and how*, and records the analytical foundation decisions that make the results defensible. Formulas in §5 are standard Basel / IFRS 9 practitioner definitions, not novel research.

1 Objective & scope

Build an end-to-end analysis that answers one business question: **which loan applicants should be approved, and what does each policy choice cost in expected loss versus earn in risk-adjusted profit?**

The project demonstrates the full analyst workflow — data cleaning, dimensional modelling, a calibrated probability-of-default model, an expected-loss framework, and a decision-support simulator — on real loan outcomes.

In scope (v1)

- Medallion data pipeline (raw → staging → analytical marts) in DuckDB, one entry-point script.
- PD model: regularised logistic regression, out-of-time validation, calibration curve, decile / KS / lift.
- Benchmark: model versus Lending Club grade alone (proves the model adds lift).
- Expected Loss = $PD \times EAD \times LGD$, with EAD and LGD as documented assumptions.
- Lending Policy Simulator (Streamlit): approval-threshold sweep → approval rate, volume, default rate, expected loss, expected profit, plus a profit-vs-approval curve.
- Star schema (DIM / FACT views) + ER diagram as a data-modelling exhibit.
- Two-page Power BI dashboard (import mode) + one-page recommendation memo.

Deferred to “future work” (documented, not built)

- Early-warning system for post-origination deterioration (requires a monthly performance panel).
- Risk-state migration matrices.
- Multi-scenario macro stress testing (the 2015–17 sample contains no downturn).

Target role: primary = credit / risk analyst (where the project is differentiated); secondary = BI analyst, carried by the SQL, star schema, dashboard and memo.

2 Foundation fixes (do these before any modelling)

A council review flagged that the results can be silently wrong unless the data foundation is hardened first. These five decisions are locked and are implemented in stage order.

2.1 Label definition & maturity window

Lending Club loans run 36 or 60 months. A loan issued near the end of the data snapshot has not had time to default, so a “not defaulted” status is *censoring*, not a good outcome. Training on immature loans biases the model and breaks calibration.

FIX

- Restrict to **36-month term loans only** (60-month loans need a 5-year window few vintages have).
- Keep only issue months old enough that essentially all loans have reached a terminal status (*Fully Paid* or *Charged Off*). Practically: issue date ≤ (snapshot date – 39 months).
- Target definition: **default = 1** if *Charged Off*, *Default*, or *Late (120+ days)* at the observation cut; **0** if *Fully Paid*. Drop loans still *Current* after the window (a documented, small exclusion).
- State the window and the status mapping verbatim in the memo’s Limitations section.

Pre-flight query (run first — it decides the cut):

```
SELECT date_trunc('month', issue_d) AS issue_month,
       count(*) AS n,
       avg( (loan_status IN ('Fully Paid','Charged Off','Default'))::int )
       AS frac_terminal
FROM raw_loans
WHERE term = ' 36 months'
```

```
GROUP BY 1 ORDER BY 1;
```

Choose the newest `issue_month` where `frac_terminal` $\geq$ 0.98. That month is the end of the modelling window; everything after it is excluded from training and test.

2.2 Feature allowlist (leakage control)

Build the feature table from an explicit **allowlist** of fields known at application time, never a denylist. Any post-origination field that leaks turns a worthless model into one that looks excellent.

EXCLUDE — post-origination (leakage)

`last_pymnt_d`, `last_pymnt_amnt`, `total_pymnt`, `total_rec_prncp`, `total_rec_int`, `recoveries`, `collection_recovery_fee`, `out_prncp`, `out_prncp_inv`, `last_fico_range_high/low`, `next_pymnt_d`, `settlement_*`, `hardship_*`, `debt_settlement_flag`, `loan_status`-derived fields

EXCLUDE — circular (Lending Club's own risk model output)

`grade`, `sub_grade`, `int_rate` — these are LC's pricing of the same risk you are predicting. Using them means predicting another model, not default. **Keep them aside as the benchmark (§2.4), not as features.**

Full origination-time feature allowlist is in §4.

2.3 Out-of-time split

- Split by **issue date**, not randomly: train on the earlier ~70% of the mature window, test on the later ~30%.
- Both partitions must sit entirely inside the maturity window from §2.1, so the test set is not systematically more immature than train.
- Report train vs test AUC / KS / Brier and the calibration curve on the **test** period only.
- Name the macro caveat: 2015–17 is a benign-credit period; the model has not seen a downturn.

2.4 Benchmark

Compare the logistic PD model against a baseline that uses **Lending Club grade alone** (`grade` → historical default rate). Report AUC and, more importantly, whether the model re-ranks borrowers within a grade. “The model adds X points of AUC and identifies the worst decile inside grade B” is the headline result.

2.5 Reproducibility across two machines

- Pin exact versions in `requirements.txt` + commit a lockfile; identical Python (3.11) on both machines.
- Do **not** hand-pass Parquet or `model.joblib` between machines as the source of truth — commit the build scripts; either machine regenerates artefacts with `python run_pipeline.py`.
- `run_pipeline.py` ends with an assertion: retraining reproduces test AUC to ± 0.005 (guards against a stale committed model).
- Windows contributor loads a sample Parquet into Power BI on **day 3** to surface DuckDB→Power BI datetime / decimal type-coercion issues early.
- The `.pbix` file is binary and un-mergeable — only the Windows contributor edits it; everyone else consumes the exported PDF.

3 System architecture

A **batch analytical pipeline** with two serving surfaces. Nothing runs continuously: there is no server and no API. DuckDB is a single file. One command rebuilds every table, the model, the scored portfolio and the exported marts.

3.1 Data flow

```
Lending Club CSV    (one pinned Kaggle mirror + snapshot date)
|    load as-is, no typing
v
[raw_]    raw_loans    <- bronze
|    SQL: cast types, parse dates, de-dupe on id, drop impossible values,
|    standardise categories, APPLY 36-month maturity window (2.1)
v
[stg_]    stg_loans    (one clean row per loan)    <- silver
|    allowlist (2.2)    |    status -> default flag (2.1)
v                    v
mart_loan_features    mart_loan_outcomes
(origination-time only)    (default 0/1)
|                    |
+----- join on id -----+
|    out-of-time split by issue_d (2.3)
v
[Python / scikit-learn]    src/train.py
    LogisticRegression pipeline: impute -> one-hot -> scale -> fit
    -> model.joblib    metrics.json    calibration_curve.png
    + one LightGBM fit -> AUC-delta paragraph
    + grade-only benchmark (2.4)
    |    load model, score every loan
    v
[Python]    src/score.py    pd_hat per loan;    EAD proxy;    LGD = 0.45
    v
    mart_loan_el    EL = pd_hat * EAD * LGD    (5.5)
    |    aggregate (SQL)
    v
    mart_portfolio_summary    mart_simulator_base
    (EL & default rate by grade /    (loan level: pd_hat, EAD, EL,
    purpose / term / vintage)    int_rate, term, balance)
    |    export /exports/*.parquet    |    read directly
    v                    v
[Power BI - Windows]    [Streamlit - Streamlit Cloud]
    Page 1    Portfolio Risk    approval-threshold + min-FICO
    Page 2    Lending Strategy    sliders -> 5 metrics + profit curve
    |
    v
    One-page recommendation memo    (+ Assumptions & Limitations, 6)
```

3.2 Technology stack

Concern	Choice	Why
Analytical store	DuckDB (single file)	Zero-config columnar SQL; reads CSV / Parquet directly; lives in the repo
Language / modelling	Python 3.11, pandas / polars, scikit-learn, lightgbm	Standard, interview-legible
Transformations	Plain .sql files run by the pipeline	Reproducible and diff-able; dbt-core optional, not required
Model development	Jupyter notebooks (import shared src/ modules)	EDA + calibration narrative; no copy-pasted logic
Simulator	Streamlit → Streamlit Community Cloud (free)	Shareable public link; note: free tier sleeps when idle
Dashboard	Power BI Desktop, import mode, over exported Parquet	Finance-role keyword; Windows-only (handled by 2-person split)

Concern	Choice	Why
Orchestration	single run_pipeline.py (or Makefile)	No Airflow / Prefect for a project this size
Version control	git + GitHub; pinned requirements + lockfile; venv	Reproducibility is table stakes

3.3 What runs where

- **Local, both machines:** python run_pipeline.py builds the DuckDB file, trains and scores the model, writes /exports/*.parquet (~2–5 min on 50–100k rows).
- **Local, either machine:** streamlit run app/simulator.py → localhost:8501.
- **Public:** GitHub repo (with a 50k-row Parquet sample + model.joblib committed for the hosted app) connected to Streamlit Community Cloud → a *.streamlit.app URL for the CV.
- **Windows machine only:** Power BI Desktop opens /exports/*.parquet, builds two pages, publishes a PDF / screenshots into the repo.
- No cloud database, no paid services. If Windows access is unavailable, Tableau Public is the substitute (runs on macOS).

3.4 Repository layout

```

credit-risk-lending-policy/
  README.md           what it is, how to run, headline results, limitations
  requirements.txt + lock pinned versions (identical on both machines)
  run_pipeline.py      raw -> marts -> model -> exports; ends with AUC-repro assert
  sql/
    01_raw.sql  02_staging.sql  03_features_outcome.sql  06_marts.sql
  src/
    features.py      the allowlist + transforms (shared by pipeline + notebooks)
    train.py         fit + calibrate + grade benchmark + metrics.json
    score.py         batch predict + expected loss
  app/simulator.py    Streamlit policy simulator
  schema/star_schema.sql  DIM_/FACT_ views (+ er_diagram.png)
  notebooks/
    01_eda.ipynb  02_features_model.ipynb  03_expected_loss.ipynb
  exports/          *.parquet marts for BI (git-ignored except 50k sample)
  reports/
    memo.md         one-pager + Assumptions & Limitations
    powerbi_dashboard.pdf exported by the Windows contributor

```

4 Feature set (origination-time allowlist)

Only these fields enter `mart_loan_features`. Each is knowable at or before the credit decision. Anything not on this list is excluded by default.

Feature	Type	Source field(s)	Notes
loan_amnt	num	loan_amnt	Requested amount
term	cat	term	Filtered to 36 months in v1
annual_inc	num	annual_inc	Log-transform; cap outliers
dti	num	dti	Debt-to-income at application
emp_length	cat	emp_length	0–10+ yrs; 'n/a' kept as its own level
home_ownership	cat	home_ownership	RENT / OWN / MORTGAGE / OTHER
purpose	cat	purpose	debt_consolidation, credit_card, ...
addr_state	cat	addr_state	Optional; watch fairness (\$6)
fico_range_low/high	num	fico_range_low/high	Application FICO band; use midpoint
earliest_cr_line	num	earliest_cr_line	Convert to credit-history months
open_acc	num	open_acc	Open credit lines
total_acc	num	total_acc	Total credit lines
revol_bal	num	revol_bal	Revolving balance
revol_util	num	revol_util	Revolving utilisation %
delinq_2yrs	num	delinq_2yrs	Delinquencies, prior 2 yrs
inq_last_6mths	num	inq_last_6mths	Credit inquiries, prior 6 mths
pub_rec	num	pub_rec	Derogatory public records
mort_acc	num	mort_acc	Mortgage accounts (if present in mirror)
verification_status	cat	verification_status	Income verification flag

Held out as benchmark, not features: grade, sub_grade, int_rate (\$2.4). **Derived:** credit_history_months = issue_d – earliest_cr_line; fico_mid = (fico_range_low + fico_range_high) / 2.

5 Formulas & metrics

Definitions follow the Basel IRB / IFRS 9 practitioner convention. Notation: for loan i , PD_i is the predicted 36-month probability of default; sums run over the approved set A under a given policy.

5.1 Probability of default (PD)

$$\text{logit}(PD_i) = b_0 + b_1 x_{i1} + \dots + b_k x_{ik} \Rightarrow PD_i = 1 / (1 + e^{-(b \cdot x_i)})$$

Regularised logistic regression (L2). Features per §4; one-hot for categoricals, standardised numerics, median imputation.

5.2 Calibration & discrimination

Brier score = $(1/N) \sum (PD_i - y_i)^2$ (lower = better; y_i is the 0/1 outcome)
Calibration curve: bin loans by predicted PD (deciles); plot mean PD_i vs observed default rate per bin; the 45° line is perfect calibration.
 $KS = \max_t |F_{\text{default}}(t) - F_{\text{non-default}}(t)|$ $AUC = P(\text{score}_{\text{default}} > \text{score}_{\text{non-default}})$
 $Gini = 2 \cdot AUC - 1$

5.3 Exposure at default (EAD)

EAD_i = current outstanding principal at the decision horizon.
v1 proxy: $EAD_i = \text{loan_amnt}_i$ (new applications) or out_prncp_i (existing loans, portfolio view only).

Assumption, stated in memo: for a 36-month amortising loan the true EAD declines over time; using the funded amount overstates EAD for seasoned loans. Acceptable for a decision-time screen; not a provisioning-grade figure.

5.4 Loss given default (LGD)

$LGD = 1 - \text{recovery rate}$. v1: $LGD = 0.45$ (fixed constant).
Optional refinement (matured, charged-off loans only, walled off from PD features):
 $\text{recovery_rate} = \sum \text{recoveries} / \sum \text{funded_amnt}(\text{charged-off}) \Rightarrow LGD = 1 - \text{recovery_rate}$

Assumption, stated in memo: a single constant LGD makes Expected Loss a monotone rescaling of PD, so EL rankings equal PD rankings. The portfolio framing adds segmentation value, not a second independent risk dimension, unless the recovery-based LGD is used.

5.5 Expected loss (EL)

$$EL_i = PD_i \times EAD_i \times LGD_i \quad \text{Portfolio EL} = \sum_{i \in A} PD_i \cdot EAD_i \cdot LGD_i$$

5.6 Lending Policy Simulator metrics

Policy = (PD cut-off τ , minimum FICO $f_{\min}$). Approved set $A = \{i : PD_i < \tau \text{ and } \text{fico_mid}_i \geq f_{\min}\}$.

Approval rate = $|A| / N$
Loan volume = $\sum_{i \in A} \text{loan_amnt}_i$
Expected default rate = $(\sum_{i \in A} PD_i) / |A|$
Expected loss (A) = $\sum_{i \in A} PD_i \cdot EAD_i \cdot LGD$
Interest income (A) = $\sum_{i \in A} \text{loan_amnt}_i \cdot \text{int_rate}_i \cdot T$ ($T = 3$ loan-years; simple, no amortisation)
Funding cost (A) = $\sum_{i \in A} \text{loan_amnt}_i \cdot r_f \cdot T$ ($r_f = 0.04$ assumed cost of funds)
Expected profit (A) = Interest income (A) – Funding cost (A) – Expected loss (A)

Horizon consistency (required)

Interest income, funding cost and expected loss are all put on the **same 3-year horizon** (multiply the two annual rate terms by $T = 3$). Mixing a lifetime loss with an annual margin is the classic error and produces a meaningless profit number. The simulator UI shows the formula and every constant ($r_f = 4\%$, $LGD = 0.45$, $T = 3$, simple interest, no prepayment, no servicing cost).

Profit-vs-approval curve: sweep τ across its range, plot Expected profit (A) against Approval rate. The peak is the profit-maximising policy; the plot is the memo's centrepiece.

5.7 Benchmark lift

Baseline: PD_i^{grade} = historical default rate of loan i 's grade (train period). Report $\Delta AUC = AUC(\text{model}) - AUC(\text{grade})$, and within-grade re-ranking: for each grade, observed default rate of the model's worst vs best PD decile.

6 Assumptions & limitations (memo section, first-class)

Stating these *is* the analyst signal. Each appears in the one-page memo.

Area	Assumption / limitation	Effect & mitigation
Label maturity	Only 36-month loans in mature vintages; Current-after-window loans dropped.	Removes censoring bias; slightly reduces sample and tilts to earlier vintages. Stated.
Reject inference / selection	Data contains only accepted, funded Lending Club loans.	PD is calibrated to approved borrowers, not the through-the-door population. The simulator extrapolates when the cut-off is loosened. Flagged explicitly; not fixable with this data.
Macro regime	2015–17 issue window contains no recession.	Out-of-regime calibration is fragile; this is why stress testing is deferred rather than faked.
LGD	Fixed 0.45 (or crude recovery-based estimate).	EL becomes a rescaling of PD unless recovery-based LGD is used. Segment analysis still valid.
EAD	Funded amount as proxy; no amortisation.	Overstates exposure for seasoned loans; fine for a decision-time screen.
Profit model	Simple interest over $T = 3$ yrs, $r_f = 4\%$, no prepayment / servicing cost.	Directional, not P&L-grade. All constants shown in the UI and memo.
Circularity	grade / sub_grade / int_rate excluded from features.	Prevents predicting LC's own model; int_rate still used in the profit calc only.
Dataset version	One pinned Kaggle mirror; column names / row dedupe vary across mirrors.	Mirror + snapshot date recorded in README; de-duplicate on id in staging.
Fairness	addr_state (and any geography / demographic proxy) is sensitive.	Either exclude addr_state or include a short disparate-impact check by state / income band.

7 Build plan & ownership

Phase	Work	Owner	Done when
Day 1–2	Pin dataset; maturity pre-flight query (§2.1); load raw → DuckDB; env parity on both machines	Mac	Maturity window chosen and written down; both machines run run_pipeline stub
Day 3	Sample Parquet → Power BI import test; star-schema draft (§3.4)	Windows	Parquet opens in Power BI with correct types
Week 1	SQL staging + DQ log; feature allowlist (§4); outcome table; out-of-time split	Mac	mart_loan_features / _outcomes built; leakage columns listed in DQ log
Week 1–2	PD model + calibration + KS/lift; grade benchmark; LightGBM delta paragraph	Mac	Test calibration curve + Δ AUC vs grade in metrics.json; AUC repro assert passes
Week 2	score.py + EL marts; segment cuts (grade / purpose / term / vintage)	Mac	mart_loan_el + mart_portfolio_summary populated
Week 2	Streamlit simulator: 2 sliders, 5 metrics, profit-vs-approval curve, assumptions panel	Windows	Deployed to Streamlit Cloud; sliders recompute; formula visible
Week 2–3	Power BI: Page 1 Portfolio Risk, Page 2 Lending Strategy; star-schema views + ER diagram	Windows	Two pages readable in 5 min; .pbix committed by Windows owner only
Week 3	One-page memo + Assumptions & Limitations (§6); README; Loom walkthrough	Both	Stranger can clone, run, and read the recommendation

If the schedule slips, cut in this order: LightGBM comparison → star schema → second Power BI page. Never cut: the maturity window, the leakage allowlist, the benchmark, the Assumptions section. Deferred items (§1) stay deferred.

8 References (practitioner, not research papers)

- Basel Committee on Banking Supervision — *International Convergence of Capital Measurement (Basel II)*, *IRB framework*: the $EL = PD \times EAD \times LGD$ definition and the regulatory meaning of each term.
- Baesens, Roesch & Scheule (2016), *Credit Risk Analytics* (Wiley): the practitioner reference for scorecard building, PD calibration, and validation used throughout this project.
- scikit-learn User Guide — *Probability calibration and model evaluation* (calibration curve, Brier score, ROC / AUC).
- IFRS 9 *Financial Instruments*, expected-credit-loss section: staging and lifetime-vs-12-month EL context (used only to frame the horizon choice).
- Lending Club data dictionary (bundled with the chosen Kaggle mirror): the authority for which columns are origination-time versus post-origination.

For a junior portfolio project these are sufficient. Academic literature (survival models, reject-inference techniques, macro overlays) is out of scope and would be scope creep — the value here is executing the standard method correctly and stating its limits.